

AWS Jam 挑战赛

题目总结与知识点解析

题目名称

CodePipeline 部署故障排查

使用服务：CodeCommit • CodePipeline • CloudFormation

难度 DevOps 中级

涉及服务 AWS CodePipeline, CodeCommit, CloudFormation

核心问题 CloudFormation 模板路径配置错误

解决方式 修正 Deploy 阶段的 Template 文件路径

目录

1 题目背景

贵公司中有人使用 CodePipeline 和 CloudFormation 建立了一条流水线用于部署 AWS 基础设施。该 CodePipeline 包含两个阶段：

- **Source 阶段：**连接一个 CodeCommit 存储库，仓库中包含一个 CloudFormation 模板文件。
- **Deploy 阶段：**使用 Source 阶段获取的模板，部署 CloudFormation 堆栈。

CloudFormation 的部署并未正常进行，Pipeline 处于**失败（Failed）**状态。你被要求作为 DevOps 顾问来排除并解决该问题。

1.1 AWS 资源清单

资源类型	说明
CodeCommit 仓库	包含 CloudFormation 模板的源代码仓库
S3 存储桶	存储 CodePipeline 运行过程中的 Artifact（工件）
IAM 角色	CodePipeline 执行所需的权限角色
CodePipeline	待修复的 CI/CD 流水线

1.2 验收标准

- CodePipeline 执行成功完成
- 新的 CloudFormation 堆栈成功部署
- CloudFormation 堆栈名称出现在主堆栈的 Outputs 中

2 故障排查过程

2.1 第一步：查看错误信息

进入 AWS 控制台 → CodePipeline → 点击失败的 Pipeline → 找到失败阶段 → 点击 **Details**。

错误信息：

```
File [template.yaml] does not exist in artifact [SourceApp]
```

错误含义解析：

- Deploy 阶段在名为 SourceApp 的 Artifact 中寻找文件 `template.yaml`
- 该文件在 Artifact 中**不存在**，导致部署失败
- 根本原因：Pipeline 配置的模板路径与仓库中实际的文件路径不匹配

2.2 第二步：定位根本原因

配置项	错误配置	正确配置
文件名	<code>template.yaml</code>	<code>working.template.yaml</code>
文件路径	根目录	<code>infrastructure-as-code/</code> 子目录
完整路径	<code>template.yaml</code>	<code>infrastructure-as-code/working.template</code>

仓库实际目录结构如下：

```
/ （仓库根目录）
  infrastructure-as-code/
    working.template.yaml    <- 实际文件位置
```

3 解决操作步骤

重要提示：在本 Jam 环境中，保存 Pipeline 修改时需要勾选“No resource updates needed for this source action change”，否则控制台可能因尝试更新变更检测相关资源而报权限错误。

这不是所有生产环境都必然需要的步骤，而是当前实验权限模型下的特殊约束。

Step 1. 进入 AWS 控制台 → CodePipeline，选择失败的 Pipeline。

Step 2. 点击右上角 **Edit**（编辑）按钮。

Step 3. 找到 **DeployTestStack** 阶段，点击 **Edit stage**。

Step 4. 点击 **GenerateChangeSet** Action 旁的编辑图标（铅笔图标）。

Step 5. 在 **Template** 部分，将 **File name** 字段修改为：

```
infrastructure-as-code/working.template.yaml
```

Step 6. 点击 **Done**（忽略顶部可能出现的 **ValidationError** 提示）。

Step 7. 点击 **DeployTestStack** 阶段的 **Done**。

Step 8. 点击顶部 **Save** 按钮。

Step 9. 弹出提示框后：

- 勾选“No resource updates needed for this source action change”
- 点击 **Save** 确认保存

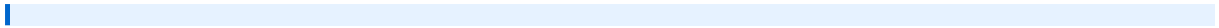
Step 10. 如果 Pipeline 未自动触发，点击 **Release Change** → 弹窗中点击 **Release**。

Artifact 路径格式说明：

在 CodePipeline Deploy 阶段配置 CloudFormation 模板路径时，格式为：

```
ArtifactName:: 相对路径/文件名.yaml
# 例如：
SourceApp::infrastructure-as-code/working.template.yaml
```

路径必须与 CodeCommit 仓库中的实际文件路径完全一致。



4 知识点解析

4.1 AWS CodePipeline 概述

4.1.1 定义

AWS CodePipeline 是一项全托管的 CI/CD（持续集成/持续交付）服务，用于自动化代码从提交到部署的完整流程。

4.1.2 核心概念

概念	说明
Pipeline（流水线）	由多个阶段组成的自动化 workflow
Stage（阶段）	流水线中的一个逻辑步骤，如 Source、Build、Deploy
Action（动作）	每个阶段中执行的具体任务
Artifact（工件）	各阶段之间传递数据的“包裹”，存储于 S3

4.1.3 典型流水线结构

```
Source --> Build --> Test --> Deploy
源码      构建      测试      部署
(CodeCommit) (CodeBuild) (测试工具) (CloudFormation/ECS)
```

本题的流水线结构（简化版）：

```
Source (CodeCommit) --> Deploy (CloudFormation)
```

4.2 CodePipeline 与相关服务的关系

服务	职责
CodeCommit	代码仓库，类似 GitHub/GitLab
CodeBuild	构建和测试代码，类似 Jenkins
CodeDeploy	将应用部署到 EC2、ECS 等计算资源
CloudFormation	基础设施即代码，自动化创建 AWS 资源
CodePipeline	编排器：串联上述服务，按顺序自动触发

关键理解： CodePipeline 本身不执行具体任务，它是一个调度器和编排器，负责按顺序触发各阶段的工具并在它们之间传递 Artifact。

4.3 CloudFormation 在 CodePipeline 中的使用

当 Deploy 阶段使用 CloudFormation 时，常见的 Action 类型有：

Action 类型	说明
Create or Update Stack	直接创建或更新 CloudFormation 堆栈
Create Change Set	生成变更集（本题使用的 GenerateChangeSet）
Execute Change Set	执行已生成的变更集
Delete Stack	删除 CloudFormation 堆栈

4.4 常见 CodePipeline 失败原因

失败阶段	常见原因	解决方案
Source	分支名称错误	核对 CodeCommit 实际分支名
Source	仓库访问权限不足	检查 IAM 角色权限
Deploy	模板路径错误	核对仓库文件实际路径（本题）
Deploy	IAM 权限不足	为 CodePipeline 角色添加 CF 权限
Deploy	S3 Artifact 桶跨区域	确保桶与 Pipeline 在同一区域

5 经验总结

5.1 排查流程

```
Pipeline 报错
|
v
查看失败阶段的 Details 错误信息
|
v
"File does not exist in artifact"?
|
v
去 CodeCommit 查看仓库实际文件结构
(或通过 S3 Artifact 桶下载 zip 查看)
|
v
确认完整相对路径 + 文件名
|
v
修改 Deploy 阶段 Template 路径
|
v
保存 (勾选无需资源更新) -> Release Change
```

5.2 最佳实践

- 文件命名规范: CloudFormation 模板使用清晰的命名, 避免与默认名称混淆
- 路径验证: 配置 Pipeline 后, 先确认 Source 阶段能成功拉取文件, 再调试 Deploy 阶段
- 权限最小化: CodePipeline IAM 角色只授予必要权限, 遵循最小权限原则
- 多环境隔离: 为 Dev/Staging/Prod 分别创建独立的 Pipeline, 避免相互干扰
- 变更集审查: 生产环境部署前使用 Change Set 预览变更内容, 再执行

5.3 本题的局限性说明

实验环境限制：本 Jam 题目限制了 `codecommit:ListRepositories` 等权限，导致正常的排查路径（浏览仓库文件结构）无法执行。这是实验环境的人为约束；在真实企业环境中，DevOps 工程师通常至少会具备以下之一：

- CodeCommit 仓库读取权限
- S3 Artifact 桶完整访问权限
- 可向开发团队询问文件结构

本题的核心考察点是：**知道在 Pipeline 的哪个位置修改模板路径**，而非独立推导出完整路径。